



Linux



File Permissions

Let's understand how Linux controls access to files 

We will learn:

- ✓ What are file permissions
- ✓ Types of permissions (r, w, x)
- ✓ Users: Owner, Group, Others
- ✓ Examples and commands



Swipe →→
to begin



2/10



What are File Permissions?



⇒ File permissions in Linux decide who can do what with a file or directory.

They control:



- Who can read the file



- Who can write or modify the file



- Who can execute the file or run a program



Permissions help to keep your system organized and secure. ✓

Example:

You may want only yourself to edit a file, but others can just read it.



Three Permission Types



① r = Read

- Allows you to view the contents of a file.



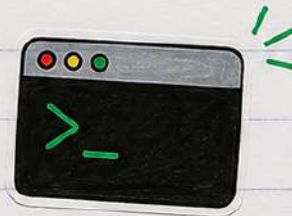
② w = Write

- Allows you to modify, edit or delete a file.



③ x = Execute

- Allows you to run a file or execute a program.



Example:

Read

Write

Execute



r

w

x



These 3 letters control what actions can be performed on a file.





Three User Categories



In Linux, permissions are set for three types of users:

① User (Owner)



The owner of the file.

② Group



Members of the file's group.

③ Others



All other users on the system.

⇒ Permissions are set separately for each category.

Example: <

rwX r-x r--

User
(Owner)

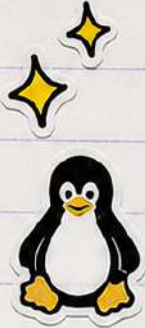
Group

Others



Each file and directory has permissions set for these three user categories.





Understanding Permission Format



⇒ Permissions are shown in a 10-character string.

Example:

- r w x r - x r - -

Breakdown:

1. Owner (User)	2. Group	3. Others
		
<u>r w x</u>	<u>r - x</u>	<u>r - -</u>
The user who owns the file.	The group that owns the file.	All other users on the system.



Note:

Each set of 3 characters (r, w, x) represents permissions for User, Group, and Others.



6/10



Numeric Permissions



⇒ Linux permissions can also be represented using numbers.

Permission Values:

r (read)		= 4
w (write)		= 2
x (execute)		= 1

How it works?

- Add the values for each permission to get the number.
- Each digit represents User, Group and Others (in order).

Common Examples:

Permissions	Breakdown	Number
rwX r-X r-X	7 5 5	755
rw- r-- r--	6 4 4	644
rwX --- ---	7 0 0	700
rwX rwX rwX	7 7 7	777

Remember:

$$4(r) + 2(w) + 1(x) = 7$$

Example:

$$rwX = 4 + 2 + 1 = 7$$



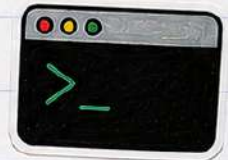
Quick Tip: Use numbers to set permissions quickly instead of writing r, w, x every time!



7/10



chmod Command



⇒ The `chmod` command is used to change file or directory permissions.

Syntax:

`chmod [options] mode filename`

① `chmod 755 file.txt` →

Gives read, write, execute permission to owner, and read, execute to group and others.



② `chmod 644 notes.txt` →

Gives read, write to owner and read only to group and others.

Quick Tip:



Use numbers (0-7) to set permissions quickly and easily!





chown Command



⇒ The chown command is used to change the owner (and group) of a file or directory.

Examples:

① Change Owner

```
chown anu file.txt
```



This command changes the owner of the file to "anu".



② Change Owner and Group

```
chown anu:dev file.txt
```



This command changes the owner to "anu" and group to "dev".



Note: You need appropriate permissions (usually root or sudo) to change ownership.





chgrp Command



➔ The `chgrp` command is used to change the group of a file or directory.

Syntax:

```
chgrp group_name file_or_directory
```

Examples:

1 Change Group

```
chgrp dev file.txt
```



This command changes the group of the file to "dev".

2 Change Group of a Directory

```
chgrp team projects/
```



This command changes the group of the directory to "team".

Quick Recap:

- ✓ Use `chgrp` to change the group.
- ✓ Only the file owner or root can change the group.
- ✓ Group must already exist on the system.



Example Scenario:

You created a file for your team. Now you want the "dev" group to access it.



```
chgrp dev file.txt
```



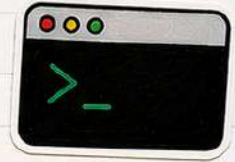
Note: Use appropriate permissions (like `chmod`) after changing the group so that the new group can read/write/execute if needed.





chmod Command

10/10



⇒ The `chmod` command is used to change the permissions of a file or directory.

Syntax:

`chmod mode file_or_directory`



Use numbers (0-7) or symbols (`r`, `w`, `x`) to set permissions.

Common Examples:

Command	Meaning	Result (Permissions)
<code>chmod 755 file.txt</code>	Read, write, execute for owner Read, execute for group Read for others	<code>rwX r-X r--</code>
<code>chmod 644 file.txt</code>	Read, write for owner Read for group and others	<code>rw- r-- r--</code>
<code>chmod 700 file.txt</code>	Read, write, execute for owner No access for group and others	<code>rwX --- ---</code>
<code>chmod 777 file.txt</code>	Read, write, execute for everyone (not recommended!)	<code>rwX rwX rwX</code>

Remember:



`r` = read (4)
`w` = write (2)
`x` = execute (1)

Practice these commands **daily** and become a Linux pro! ✨



FOLLOW

For more Linux tips, commands and notes!

